Trappole d'esame — Dominio 2: Progettazione degli strumenti e integrazione MCP

> **Come usare questo file.** Per ogni trappola: risposta sbagliata (❌), perché, risposta corretta (✅), segnale nella domanda (🔎). ⚠️ **Guida vs docs** segnala dove la guida d'esame e la documentazione attuale (ottobre 2026) divergono: all'esame vale la guida.

Regole d'oro del dominio

1. **La descrizione dello strumento è la prima leva**: se il modello sceglie lo strumento sbagliato, migliora le descrizioni prima di tutto il resto.
2. **L'esame premia il primo passo a basso sforzo e alto impatto**, non la soluzione architetturale più elaborata.
3. **Errori strutturati**: `errorCategory` + `isRetryable` + messaggio leggibile. Un risultato vuoto *non è* un errore.
4. **Pochi strumenti per agente, mirati e vincolati** (minimo privilegio).
5. **MCP: `.mcp.json` nel progetto per il team, `~/.claude.json` per l'utente; segreti via variabili d'ambiente.**
6. **Grep cerca nel contenuto, Glob cerca nei percorsi; Edit prima di Read + Write.**

---

1. Progettazione dell'interfaccia degli strumenti

Scenario tipico: due strumenti con descrizioni minime (`get_customer` / `lookup_order`) e il modello sceglie quello sbagliato. Ordine di intervento atteso:

1. **Espandere le descrizioni** (cosa fa, quando usarlo, quando *non* usarlo, formato input/output, differenza dagli strumenti simili);
2. ricontrollare il system prompt;
3. solo dopo, eventuali interventi architetturali.

Trappola 1 — Few-shot per correggere il routing sbagliato causato da descrizioni scarne

Perché è sbagliato: gli esempi aggiungono token senza toccare la causa: il modello è confuso perché le descrizioni non distinguono gli strumenti.

✅ Correggi prima le **descrizioni**.

Trappola 2 — Un classificatore di routing come primo intervento

Perché è sbagliato: è sovra-ingegnerizzato; aggiunge infrastruttura e scavalca la comprensione del linguaggio del modello.

✅ Espandi le descrizioni: affronta direttamente la causa.

Trappola 3 — Fondere strumenti simili in uno solo come primo intervento

Perché è sbagliato: il consolidamento può essere una buona scelta a lungo termine, ma costa più che riscrivere le descrizioni.

✅ Primo passo: descrizioni. 🔎 Se la domanda chiede "il **primo** passo" o "il più efficace con il minimo sforzo", scegli l'intervento più leggero che colpisce la causa.

Trappola 4 — Dimenticare il system prompt dopo aver aggiornato le descrizioni

Perché è sbagliato: istruzioni nel system prompt basate su parole chiave (es. "quando l'utente nomina un *ordine*, cerca il cliente") possono prevalere silenziosamente su descrizioni ben scritte e creare associazioni indesiderate.

✅ Dopo ogni modifica alle descrizioni, **rileggi il system prompt** e riformula le frasi che spingono verso uno strumento specifico.

---

2. Risposte di errore strutturate

Formato di riferimento:

```
{
  "isError": true,
  "errorCategory": "transient | validation | business | permission",
  "isRetryable": false,
  "message": "Il rimborso di 600 € supera il limite di policy di 500 €"
}
```

Categoria	`isRetryable`	**Come recupera l'agente**
Transitorio (timeout, 503)	`true`	Ripete la stessa chiamata
Validazione (input malformato)	`false`	Corregge l'input e fa una **nuova** chiamata
Business (violazione di policy)	`false`	Percorso alternativo / escalation
Permessi	`false`	Escalation o soggetto diverso con i permessi

Trappola 5 — Riprovare quando una query riuscita restituisce zero risultati

Perché è sbagliato: risultato vuoto da una query andata a buon fine = "nessun dato corrisponde". Riprovando si ottiene lo stesso vuoto.

✅ Accetta il risultato e rispondi di conseguenza ("non ho trovato ordini con quel numero").

Trappola 6 — Messaggi generici ("Operazione fallita") senza metadati

Perché è sbagliato: l'agente non distingue un problema transitorio da una violazione di regole di business, quindi non sa come recuperare.

✅ Restituisci `errorCategory`, `isRetryable` e una descrizione leggibile.

Trappola 7 — Trattare gli errori di business come ritentabili

Perché è sbagliato: "il rimborso supera il limite" fallirà identico a ogni tentativo.

✅ `isRetryable: false` → percorso alternativo, tipicamente **escalation**.

Trappola 8 — Segnare un errore di validazione `isRetryable: true` "perché l'agente può recuperare"

Perché è sbagliato: `isRetryable` risponde a una domanda precisa: *ripetere esattamente questa chiamata può funzionare?* Un ID ordine malformato fallisce sempre lo stesso controllo.

✅ Validazione → `isRetryable: false`. L'agente recupera correggendo l'input in una **nuova** chiamata, guidato da `errorCategory: "validation"`.

Trappola 9 — Leggere `isRetryable: false` come "abbandona il task"

Perché è sbagliato: nessuna delle categorie non ritentabili significa arrendersi:

- validazione → l'agente corregge l'input da solo;
- business → percorso alternativo o escalation;
- permessi → escalation o un soggetto con i permessi giusti.

✅ `false` significa solo "non ripetere la chiamata identica".

Trappola 10 — Un sottoagente che nasconde gli errori restituendo un risultato vuoto come successo

Perché è sbagliato: il coordinatore non può distinguere "non ho trovato nulla" da "non sono riuscito a cercare".

✅ Restituisci un contesto di errore strutturato: **tipo di errore, query tentata, eventuali risultati parziali** (vedi anche D5, trappole 9-11).

---

3. Distribuzione e scelta degli strumenti

Trappola 11 — Far passare dal coordinatore tutte le verifiche semplici quando l'85% sono lookup banali

Perché è sbagliato: ogni passaggio dal coordinatore aggiunge 2-3 salti per richiesta.

✅ Dai all'agente di sintesi uno strumento **mirato** (`verify_fact`) per gestire direttamente l'85% dei casi semplici; i casi complessi continuano a passare dal coordinatore. Risultato: latenza ridotta fino a circa il 40%.

🔎 Una percentuale alta di casi semplici e ripetitivi suggerisce di dare uno strumento con ambito ristretto direttamente all'agente che ne ha bisogno.

Trappola 12 — `tool_choice: "auto"` quando serve per forza un output strutturato

Perché è sbagliato: con `auto` il modello può rispondere in testo libero invece di chiamare lo strumento.

`tool_choice`	**Effetto**
`auto`	il modello decide se usare uno strumento
`any`	deve usare **uno** strumento qualsiasi
`{"type": "tool", "name": "..."}`	deve usare **quello specifico**
`none`	nessuno strumento

✅ Guida d'esame: `any` garantisce una chiamata; lo strumento forzato garantisce quello specifico.

⚠️ **Guida vs docs:** secondo la documentazione attuale alcuni modelli recenti rifiutano `any` e lo strumento forzato e raccomandano `auto` con `strict: true`. All'esame rispondi secondo la guida.

Trappola 13 — Dare 18 strumenti a un solo agente aspettandosi una scelta affidabile

Perché è sbagliato: ogni strumento in più aumenta la complessità della decisione e l'affidabilità cala.

✅ Dividi gli strumenti **per ruolo** in set piccoli (la guida contrappone 18 a 4-5 per agente).

⚠️ **Guida vs docs:** la documentazione attuale colloca il degrado tra 30 e 50 strumenti. All'esame: pochi strumenti per agente.

Trappola 14 — Dare a un sottoagente un generico `fetch_url` per caricare documenti

Perché è sbagliato: uno strumento generico consente usi impropri (qualsiasi URL, anche interno o malevolo).

✅ Uno strumento vincolato, ad esempio `load_document` che accetta solo URL di documenti validi: **minimo privilegio** e scopo più chiaro.

---

4. Integrazione con server MCP

File	**Ambito**	**Versionato/condiviso?**
`.mcp.json` (radice del progetto)	team / progetto	✅ sì, via git
`~/.claude.json`	utente	❌ personale

Trappola 15 — Costruire un server MCP custom per un'integrazione standard (Jira, GitHub…)

Perché è sbagliato: esistono già server MCP della community per le integrazioni comuni.

✅ Valuta prima quelli esistenti; un server custom si giustifica solo per **workflow specifici del team** che i server esistenti non coprono.

Trappola 16 — Mettere la configurazione MCP del team in `~/.claude.json`

Perché è sbagliato: è un file personale, non versionato: i colleghi non lo ricevono.

✅ `.mcp.json` nella radice del progetto.

Trappola 17 — Incollare un token in `.mcp.json` "tanto il repo è privato"

Perché è sbagliato: un repo privato non è un archivio di segreti: ogni clone e ogni fork si porta il token nella cronologia git.

✅ Espansione di variabili d'ambiente:

```
{ "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" } }
```

Ogni sviluppatore imposta il token localmente.

Trappola 18 — Una regola nel system prompt, o la rimozione degli strumenti integrati, per far usare all'agente uno strumento MCP

Perché è sbagliato: la regola nel prompt è probabilistica; rimuovere gli strumenti integrati rompe il lavoro per cui servono ancora. Il vero motivo è che l'agente sceglie leggendo le descrizioni, e una descrizione MCP scarna perde contro quella dettagliata di uno strumento integrato.

✅ **Espandi la descrizione dello strumento MCP**: capacità, quando preferirlo, cosa restituisce.

---

5. Strumenti integrati (Claude Code)

Strumento	Cerca in	Esempio
Grep	**contenuto** dei file	chi chiama `processPayment()`?
Glob	**percorsi/nomi** dei file	tutti i `**/*.test.ts`
Read	un file specifico	leggere il punto d'ingresso trovato
Edit	sostituzione mirata di testo	cambiare una riga
Write	file intero	creare o riscrivere tutto

Trappola 19 — Glob per trovare chi chiama una funzione

Perché è sbagliato: Glob confronta solo i percorsi, non guarda dentro i file.

✅ **Grep** per nomi di funzioni, import, messaggi di errore.

Trappola 20 — Grep per trovare file per estensione o nome

Perché è sbagliato: Grep tecnicamente può trovare nomi di file citati nel contenuto, ma lo strumento progettato per i percorsi è Glob.

✅ **Glob** per pattern di percorso (file di test, file di configurazione).

Trappola 21 — Leggere tutti i file sorgente all'inizio

Perché è sbagliato: consuma il budget di contesto prima ancora di sapere cosa serve.

✅ Approccio **incrementale**: Grep per trovare i punti d'ingresso → Read per seguire il flusso da lì.

Trappola 22 — Read + Write come default per ogni modifica

Perché è sbagliato: Read + Write carica e riscrive tutto il file; Edit tocca solo il testo interessato ed è più veloce e leggero.

✅ Prima **Edit**; Read + Write come ripiego quando Edit non trova un ancoraggio univoco.

Trappola 23 — Allargare `old_string` o usare `replace_all` quando l'ancoraggio non è univoco ma va cambiata una sola occorrenza

Perché è sbagliato:

- allargare `old_string` funziona solo se esiste un contesto univoco da includere;
- `replace_all` cambia **tutte** le occorrenze, non una.

✅ Guida d'esame: se Edit non trova un ancoraggio univoco → **Read + Write**.

⚠️ **Guida vs docs:** la documentazione attuale suggerisce di allargare `old_string` o usare `replace_all`. Ma se la domanda specifica che l'ancoraggio non può essere reso univoco *e* va cambiata una sola occorrenza, nessuna delle due funziona → Read + Write.